



**INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE COMERCIO Y ADMINISTRACIÓN
UNIDAD SANTO TOMÁS
LICENCIATURA EN NEGOCIOS DIGITALES**



INFORME DE PRÁCTICA #7

Desarrollo tienda de teléfonos

Checkout y Finalización de Compra

Materia: Laboratorio Empresarial

Maestro: Ing. Jovan Del Prado López

Estudiantes: Edna Escobar Hernández, Moreno Ruiz Diana Yael

Grupo: 6GM1

1. Introducción

La presente práctica corresponde a la séptima entrega del proyecto Tienda de Teléfonos, desarrollado bajo el patrón de arquitectura Modelo–Vista–Controlador (MVC) con PHP y MySQL. En las prácticas anteriores se construyó el sistema completo de autenticación, catálogo de productos y carrito de compras con actualización y eliminación de ítems.

En esta séptima práctica se implementa el proceso de checkout, que representa la etapa final del flujo de compra. Al ejecutarse, el sistema verifica que exista stock suficiente para cada producto en el carrito, descuenta las cantidades compradas del inventario, vacía el carrito del usuario y muestra un mensaje de confirmación. Si algún producto no tiene stock suficiente, se cancela toda la operación y se notifica al usuario.

El trabajo se divide en cuatro partes: agregar el método checkout() al CartController junto con la propiedad productModel, incorporar el método updateStock() al modelo Product, registrar la nueva ruta en el enrutador index.php y realizar pruebas completas del flujo mediante el archivo test_checkout.php.

Esta práctica cierra el ciclo de compra del proyecto, integrando por primera vez dos modelos dentro de un mismo controlador y asegurando la integridad del inventario mediante una validación previa al procesamiento.

2. Objetivos

Objetivo General

Implementar el proceso completo de checkout en el sistema tienda_telefonos, verificando el stock disponible antes de confirmar la compra, actualizando el inventario de productos, vaciando el carrito y mostrando mensajes de confirmación al usuario, mediante el patrón MVC en PHP.

Objetivos Específicos

- Agregar la propiedad \$productModel al CartController e inicializarla en el constructor para permitir el acceso al modelo de productos desde el controlador del carrito.
- Implementar el método checkout() que valide el stock de cada ítem antes de procesar la compra y cancele la operación con mensaje de error si algún producto no tiene existencias suficientes.
- Agregar el método updateStock() al modelo Product.php que actualice el inventario con una consulta SQL segura, descotando solo si el stock disponible es mayor o igual a la cantidad solicitada.
- Registrar la ruta checkout en el switch del enrutador index.php con verificación de autenticación previa.
- Verificar el flujo completo de checkout mediante el archivo test_checkout.php, comprobando el estado del stock antes y después de la compra.

3. Desarrollo Paso a Paso

Parte 1 — Agregar método checkout al CartController (15 min)

Se abre el archivo controllers/CartController.php. Al inicio del archivo se agrega el require_once del modelo Product. Dentro de la clase se declara la propiedad privada \$productModel y se inicializa en el constructor junto al \$cartModel ya existente.

A continuación se agrega el método checkout(). Su lógica opera en dos fases: primero recorre todos los ítems del carrito y consulta el stock actual de cada producto; si alguno tiene stock insuficiente, guarda un mensaje de error en sesión y redirige al carrito sin procesar nada. Si todos tienen stock suficiente, en el segundo recorrido llama a updateStock() por cada ítem y luego limpia el carrito con clearCart(), redirigiendo al dashboard con mensaje de éxito.

```
// controllers/CartController.php - Cambios de la Práctica 7

require_once 'models/Product.php'; // Importar modelo de productos

class CartController {
    private $cartModel;
    private $productModel; // Nueva propiedad para manejar el inventario

    public function __construct() {
        $this->cartModel = new Cart();
        $this->productModel = new Product(); // Se inicializa junto al cartModel
    }

    // ... métodos existentes (addToCart, removeFromCart, updateCart) ...

    // Procesa la compra final: valida stock, actualiza inventario y limpia carrito
    public function checkout() {
        $user_id = $_SESSION['user_id'];
        $cartItems = $this->cartModel->getCartItems($user_id);

        // FASE 1: Verificar stock de todos los productos ANTES de procesar
        foreach($cartItems as $item) {
            $product = $this->productModel->getById($item['producto_id']);
            if($product['stock'] < $item['cantidad']) {
                // Si hay insuficiencia cancela y avisa al usuario
                $_SESSION['error'] = '✖ Stock insuficiente para: ' .
                    $item['nombre'];
                header('Location: index.php?action=cart');
                exit();
            }
        }

        // FASE 2: Descontar stock producto por producto
        foreach($cartItems as $item) {
            $this->productModel->updateStock($item['producto_id'],
            $item['cantidad']);
        }

        // Limpiar el carrito del usuario tras la compra exitosa
        $this->cartModel->clearCart($user_id);
        $_SESSION['success'] = '🎉 ¡Compra realizada con éxito! Gracias por tu
        compra.';
        header('Location: index.php?action=dashboard');
        exit();
    }
}
```

Parte 2 — Agregar método updateStock al modelo Product (10 min)

Se abre el archivo models/Product.php y se agrega el método updateStock(). Este método recibe el ID del producto y la cantidad a descontar. Prepara una consulta UPDATE con PDO que reduce

el stock solo si el valor actual es mayor o igual a la cantidad solicitada (WHERE stock >=:cantidad), evitando que el inventario quede en negativo.

```
// models/Product.php - Nuevo método

// Descuenta la cantidad comprada del stock del producto
// La cláusula AND stock >= :cantidad evita inventario negativo
public function updateStock($id, $cantidad) {
    $query = 'UPDATE ' . $this->table_name . '
        SET stock = stock - :cantidad
        WHERE id = :id AND stock >= :cantidad';

    $stmt = $this->conn->prepare($query);
    $stmt->bindParam(':id', $id); // ID del producto a actualizar
    $stmt->bindParam(':cantidad', $cantidad); // Unidades vendidas
    return $stmt->execute(); // Retorna true si se ejecutó
}
```

Parte 3 — Agregar ruta en index.php (5 min)

Se localiza el switch de acciones en index.php y se agrega el caso checkout. Al igual que las rutas del carrito, primero verifica la autenticación del usuario antes de delegar al controlador.

```
// index.php - Nueva ruta en el switch de acciones

// Ruta para finalizar la compra
case 'checkout':
    $authController->checkAuth(); // Verifica que el usuario esté logueado
    $cartController->checkout(); // Delega la lógica al controlador
    break;
```

Parte 4 — Prueba completa del flujo (30 min)

Se crea el archivo test_checkout.php en la raíz del proyecto para probar el flujo completo de forma aislada. El script inicia sesión con un usuario de prueba, consulta el stock del producto con ID 1 antes de la compra, agrega ese producto al carrito, ejecuta el proceso de checkout (actualización de stock y limpieza del carrito), y vuelve a consultar el stock para confirmar que disminuyó correctamente.

```
<?php
// test_checkout.php - Script de prueba del flujo de checkout
session_start();
require_once 'models/User.php';
require_once 'models/Product.php';
require_once 'models/Cart.php';

echo '<h1>🔑 Prueba de Checkout</h1>';

// 1. Autenticar usuario de prueba
$userModel = new User();
$user = $userModel->login('admin@test.com', 'admin123');

if($user) {
    $_SESSION['user_id'] = $user['id'];
    $cartModel = new Cart();
```

```

$productModel = new Product();

// 2. Consultar stock ANTES de la compra
$producto_prueba = $productModel->getById(1);
echo '<h3>📦 Stock antes de la compra:</h3>';
echo 'Producto: ' . $producto_prueba['nombre'] . '<br>';
echo 'Stock: ' . $producto_prueba['stock'] . '<br><br>';

// 3. Agregar producto al carrito (ID producto=1, cantidad=1)
$cartModel->addToCart($user['id'], 1, 1);
echo '✅ Producto agregado al carrito<br>';

// 4. Obtener ítems y ejecutar el checkout manualmente
$items = $cartModel->getCartItems($user['id']);
echo 'Items en carrito: ' . count($items) . '<br>';

foreach($items as $item) {
    $productModel->updateStock($item['producto_id'], $item['cantidad']);
}
$cartModel->clearCart($user['id']);
echo '✅ Checkout completado<br>';

// 5. Consultar stock DESPUÉS de la compra para verificar descuento
$producto_despues = $productModel->getById(1);
echo '<h3>📦 Stock después de la compra:</h3>';
echo 'Producto: ' . $producto_despues['nombre'] . '<br>';
echo 'Stock: ' . $producto_despues['stock'] . '<br>';

echo '<h3>✅ Prueba exitosa! El stock se actualizó correctamente.</h3>';
}
?>

```

Para ejecutar la prueba se accede desde el navegador a:
http://localhost/tienda_telefonos/test_checkout.php

- Se verificó que el número de stock antes de la compra disminuyó en 1 (o en la cantidad comprada) después del checkout.
- Se confirmó que el carrito quedó vacío tras la compra consultando getCartItems().
- Se probó el caso de stock insuficiente modificando temporalmente el stock a 0 en la base de datos y verificando que el sistema devuelve el mensaje de error y no procesa la compra.
- Se comprobó que sin sesión activa el acceso a la ruta checkout redirige al login correctamente.

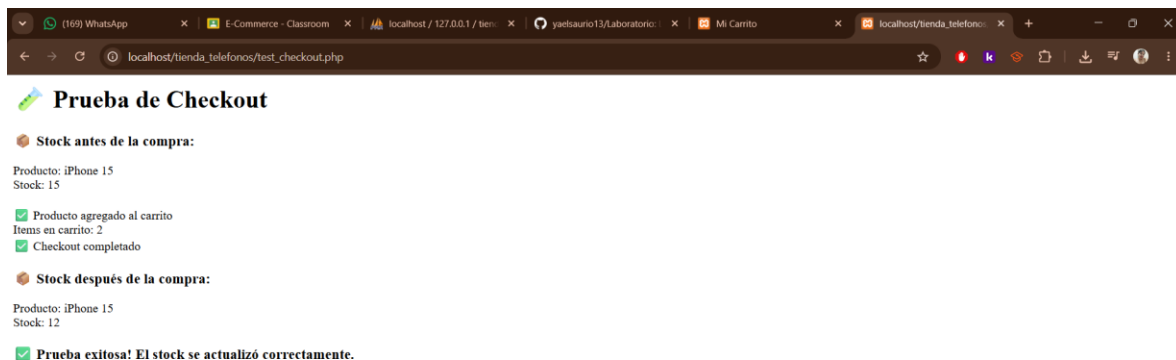
4. Tabla de Pruebas

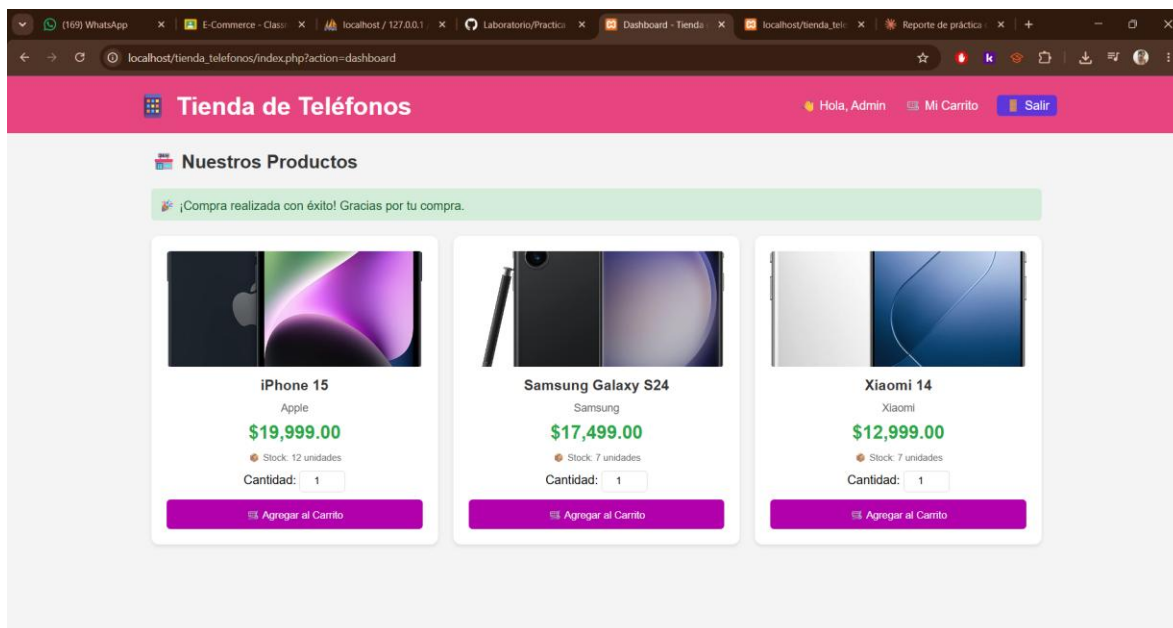
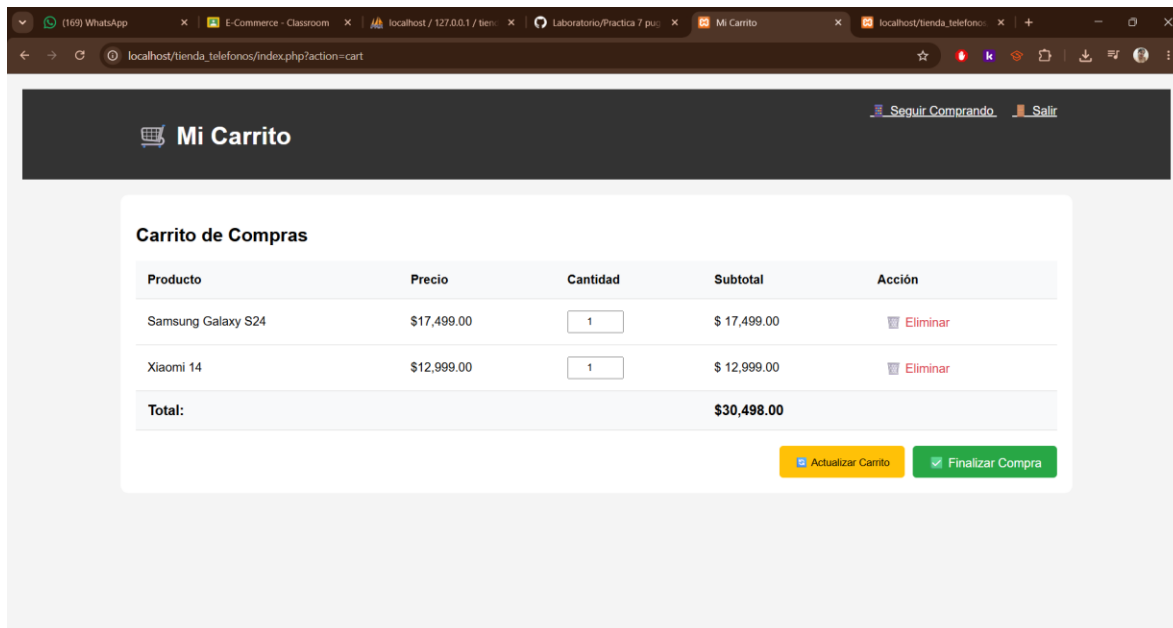
#	Criterio de Éxito	Acción Realizada	Resultado Esperado	Estado
1	El checkout verifica el stock disponible	Intentar finalizar la compra con un producto cuyo stock es 0	El sistema muestra mensaje de error y redirige al carrito sin procesar	✅

2	El stock se actualiza después de la compra	Realizar checkout con productos en el carrito y verificar la BD	El campo stock de cada producto disminuye en la cantidad comprada	✓
3	El carrito se vacía después del checkout	Completar una compra exitosa y revisar el carrito	El carrito aparece vacío al regresar a la vista del carrito	✓
4	Se muestra mensaje de confirmación	Completar una compra sin errores de stock	Aparece el mensaje de éxito en el dashboard tras la redirección	✓
5	No permite comprar más del stock disponible	Agregar cantidad mayor al stock disponible e intentar checkout	Se detiene la operación con mensaje de stock insuficiente por producto	✓

5. Capturas de Pantalla

A continuación, se presentan tres capturas del sistema en ejecución que evidencian el correcto funcionamiento del proceso de checkout implementado en esta práctica.





6. Conclusiones

Con la Práctica 7 se completó el flujo de compra del proyecto tienda_telefonos. La implementación del método checkout() en el CartController integró por primera vez dos modelos dentro de un mismo controlador, demostrando que el patrón MVC permite escalar la complejidad del sistema sin romper la separación de responsabilidades.

Un aspecto técnico destacable fue la estrategia de validación en dos fases: primero revisar el stock de todos los productos antes de modificar cualquier dato, y solo entonces proceder con las actualizaciones. Este enfoque evita inconsistencias en la base de datos (por ejemplo, que se

descuento el stock de los primeros productos, pero no de los últimos si uno falla), lo cual es fundamental en sistemas transaccionales.

El método `updateStock()` en el modelo `Product.php` incorpora una cláusula `WHERE stock >=:` cantidad que actúa como segunda barrera de seguridad a nivel de base de datos. Aunque el controlador ya verifica el stock antes de llamar al método, esta condición en SQL garantiza que ninguna actualización deje el inventario en valores negativos, incluso ante condiciones de carrera o llamadas directas al modelo.

La creación del archivo `test_checkout.php` resultó útil para validar el comportamiento del sistema de forma aislada, sin pasar por la interfaz gráfica. Este tipo de scripts de prueba directa acelera el proceso de depuración y es una práctica recomendada en el desarrollo web antes de integrar funcionalidades en el flujo principal.

Como aprendizaje general, esta práctica consolidó la comprensión del ciclo completo de una tienda en línea: desde el inicio de sesión y la navegación por el catálogo, hasta la gestión del carrito y la finalización de la compra con actualización de inventario. El proyecto `tienda_telefonos` ahora cuenta con una arquitectura MVC funcional de extremo a extremo.